

APUNTE 4

Protección de Memoria y Seguridad del Kernel

La protección de memoria es un pilar fundamental en la seguridad de los sistemas operativos modernos. Su objetivo es prevenir la explotación de vulnerabilidades como buffer overflows o use-after-free, que permiten a atacantes tomar control de procesos o incluso del kernel.

El esquema general es de defensa en profundidad: ninguna mitigación es perfecta por sí sola, pero cada capa eleva el costo del ataque. Un atacante que logra evadir ASLR todavía enfrenta DEP/NX. Si evade DEP con ROP, todavía enfrenta Stack Canaries. Si tiene un exploit funcional, Seccomp restringe lo que puede hacer. La combinación de todas las capas hace los ataques significativamente más complejos y costosos.

Mitigaciones de Memoria

Las mitigaciones de memoria son mecanismos implementados por el sistema operativo, el compilador y el hardware para dificultar la explotación de vulnerabilidades como buffer overflows. Se aplican a nivel de proceso y protegen el espacio de memoria de usuario.

1.1 Buffer Overflow — Motivación para las Mitigaciones

Antes de estudiar las defensas, es necesario entender qué se está protegiendo. Un buffer overflow ocurre cuando un programa escribe más datos en un buffer (una región de memoria) de lo que este puede contener. Los bytes adicionales sobrescriben memoria adyacente en el stack, lo que puede corromper el estado del programa o permitir a un atacante modificar la dirección de retorno de una función para ejecutar código malicioso.

Ejemplo: código vulnerable con gets()

El siguiente programa en C contiene una vulnerabilidad clásica de buffer overflow:

```
// CÓDIGO VULNERABLE — buffer overflow con gets()
#include <stdio.h>
#include <string.h>

void vuln() {
    char buf[8];           // buffer de solo 8 bytes
    gets(buf);             // PELIGROSO: no verifica el tamaño
    printf("Ingresaste: %s\n", buf);
}

int main() {
    vuln();
    return 0;
}
```

¿Por qué este código es vulnerable?

- La función `gets(buf)` lee una línea completa de entrada desde `stdin` sin verificar el tamaño del buffer.
- `gets()` no limita cuántos caracteres lee. Si el usuario ingresa más de 8 caracteres, los bytes adicionales sobrescriben memoria adyacente al buffer en el stack.
- El stack contiene, justo después del buffer, variables locales, el EBP guardado y la dirección de retorno de la función `vuln()`.
- Un atacante que controla el input puede escribir exactamente en la posición de la dirección de retorno y hacer que el programa salte a cualquier dirección de memoria que él elija.

¿Cómo se ve esto en el stack?

```
# Layout del stack durante la ejecución de vuln()
Dirección alta
| Dirección de retorno (4/8 bytes) | ← objetivo del atacante
| EBP guardado (4/8 bytes)       |
| uf[8] ← buffer vulnerable     | ← overflow empieza aquí
Dirección baja

# Con input de 8 bytes: llena el buffer, todo normal
# Con input de 12 bytes: sobrescribe EBP
# Con input de 16 bytes (x86): sobrescribe dirección de retorno
# El atacante controla a dónde vuelve la función
```

Solución: reemplazar `gets()` por `fgets()`

La función `fgets()` permite especificar explícitamente el tamaño máximo del buffer, previniendo el overflow:

```
// CÓDIGO SEGURO - usando fgets() con límite de tamaño
void vuln_segura() {
    char buf[8];
    fgets(buf, sizeof(buf), stdin); // lee como máximo 7 chars + \0
    printf("Ingresaste: %s\n", buf);
}

# fgets(buffer, tamaño_máximo, stream)
# Lee a lo sumo tamaño_máximo-1 caracteres y agrega \0 al final
# Nunca sobrescribe más allá del tamaño indicado
# Completamente seguro contra buffer overflow
```

`gets()` fue eliminada del estándar C11 por ser fundamentalmente insegura. Ningún compilador moderno debería usarla. Funciones seguras: `fgets()` (lectura con límite), `strncpy()` en lugar de `strcpy()`, `sprintf()` en lugar de `sprintf()`. El compilador GCC emite una advertencia cuando detecta el uso de `gets()`.

El resto de esta unidad explica los mecanismos que el sistema operativo y el compilador implementan para que, aun cuando el programador cometa este tipo de errores, el atacante encuentre barreras que dificulten o impidan la explotación.

1.2 DEP / NX Bit — Data Execution Prevention

DEP (Data Execution Prevention) o NX Bit (No-eXecute) es una protección implementada a nivel de hardware que marca ciertas regiones de memoria como no ejecutables. El principio es simple: ninguna región de memoria debería ser al mismo tiempo escribible y ejecutable.

Sin DEP: el atacante puede inyectar shellcode en un buffer (región escribible) y luego redirigir la ejecución a ese buffer. El CPU ejecuta el shellcode normalmente.

Con DEP: el stack y el heap están marcados como no ejecutables. Cuando el CPU intenta ejecutar código en esas regiones, genera un fault y el proceso es terminado.

```
# Verificar si NX está habilitado en un binario
$ checksec --file=./programa
RELRO      STACK CANARY      NX            PIE
Full RELRO    Canary found  NX enabled    PIE enabled

# Compilar con NX habilitado (por defecto en GCC moderno)
$ gcc -z noexecstack programa.c -o programa

# Compilar SIN NX (solo para pruebas/educación)
$ gcc -z execstack programa.c -o programa_vulnerable

# Ver el layout de memoria de un proceso en ejecución
$ cat /proc/$(pidof programa)/maps
7f3a1000-7f3a2000 r-xp /lib/libc.so.6 ← ejecutable, no escribible
7f3a2000-7f3a3000 rw-p /lib/libc.so.6 ← escribible, no ejecutable
7fff0000-7fff1000 rw-p [stack]      ← stack: rw, sin x
```

DEP se implementa usando el bit NX presente en CPUs Intel desde 2004 (llamado XD — Execute Disable) y AMD (llamado Enhanced Virus Protection). El sistema operativo configura este bit en las tablas de páginas para cada región de memoria. Sin soporte de hardware, DEP no puede implementarse eficientemente.

Caso real — CodeRed (2001): el gusano CodeRed explotaba un desbordamiento de buffer en servidores IIS de Microsoft. Inyectaba shellcode directamente en la pila y lo ejecutaba desde allí. En sistemas modernos con NX Bit habilitado, ese ataque sería bloqueado porque el CPU rechazaría ejecutar código en una región de stack marcada como no ejecutable.

1.3 ASLR — Address Space Layout Randomization

ASLR aleatoriza las direcciones base donde se cargan el stack, el heap, las bibliotecas compartidas y el código del ejecutable en cada ejecución del proceso. Su objetivo es que un atacante no pueda predecir la ubicación de código o datos en memoria.

ASLR es una técnica de seguridad implementada en sistemas operativos como Linux, Windows y macOS. Cuando un proceso se ejecuta con ASLR activado, los siguientes segmentos se cargan en direcciones distintas en cada ejecución:

- Stack (Pila)
- Heap (Montículo)
- Bibliotecas compartidas (ejemplo: libc)
- Código del binario ejecutable (cuando se compila con PIE)

¿Cómo ayuda ASLR a mitigar ataques?

Los ataques como buffer overflow y ROP dependen de conocer direcciones de memoria específicas para sobrescribir punteros o reutilizar código ya existente en bibliotecas. Sin ASLR, las direcciones son predecibles y constantes entre ejecuciones, lo que permite al atacante calcular exactamente dónde inyectar su código.

```
# Verificar el estado de ASLR en Linux
$ cat /proc/sys/kernel/randomize_va_space
2          ← 0=desactivado, 1=parcial, 2=completo (valor recomendado)

# — SIN ASLR — dirección del buffer es siempre la misma
$ sudo sysctl -w kernel.randomize_va_space=0
$ ./vuln
Buffer: 0x7fffffffddcc0 ← constante
$ ./vuln
Buffer: 0x7fffffffddcc0 ← exactamente igual, predecible
$ ./vuln
Buffer: 0x7fffffffddcc0 ← el atacante puede hardcodear esta dirección

# — CON ASLR — la dirección cambia en cada ejecución
$ sudo sysctl -w kernel.randomize_va_space=2
$ ./vuln
Buffer: 0x7ffd78e3da70 ← aleatoria
$ ./vuln
Buffer: 0x7ffe12a4b390 ← diferente
$ ./vuln
Buffer: 0x7ffb98c1e5f0 ← diferente en cada ejecución

# El exploit que funcionaba con ASLR desactivado ahora falla:
# la dirección hardcodeada apunta a memoria inválida → Segfault
```

Comparación de exploits en sistemas con y sin ASLR

Un exploit basado en buffer overflow puede funcionar en un sistema sin ASLR pero fallará en un sistema con ASLR. La siguiente tabla resume la diferencia:

Característica	Sin ASLR	Con ASLR
Dirección del buffer	Constante (0x7fffffffddcc0)	Aleatoria en cada ejecución
Dirección de libc	Fija y conocida	Aleatorizada cada vez
Exploit con dirección fija	Funciona de forma confiable	Falla: la dirección no corresponde
Bruteforce posible	No es necesario	A veces posible si el espacio es pequeño
Requiere info leak para explotar	No	Sí — necesita filtrar una dirección real

ASLR no es una protección completa. Si el atacante puede leer el valor de una dirección en memoria (info leak) — por ejemplo a través de una vulnerabilidad de format string — puede calcular la base real y reconstruir la dirección del objetivo. Por eso ASLR debe combinarse con otras mitigaciones

1.4 PIE — Position Independent Executable

ASLR por defecto aleatoriza el stack, el heap y las bibliotecas compartidas, pero no necesariamente la sección de texto (código) del ejecutable. Para aleatorizar también el código del binario, este debe compilarse como Position Independent Executable (PIE). Sin PIE, incluso con ASLR activo, las funciones del programa tienen direcciones fijas y conocidas.

```
# Compilar SIN PIE — el código siempre carga en la misma dirección
$ gcc -no-pie programa.c -o programa_nopie
$ readelf -h programa_nopie | grep Type
Type: EXEC (Executable file) ← dirección base fija

# Compilar CON PIE — el código también se aleatoriza con ASLR
$ gcc -fPIE -pie programa.c -o programa_pie
$ readelf -h programa_pie | grep Type
Type: DYN (Shared object file) ← puede cargarse en cualquier base

# Con PIE, incluso la dirección de main() cambia
$ nm programa_pie | grep main
0000000000001149 T main ← offset relativo, no dirección absoluta
```

GCC moderno compila con PIE habilitado por defecto en la mayoría de las distribuciones Linux. Un exploit que usa la dirección de una función del binario para ROP necesita primero una info leak para conocer la base real del ejecutable.

1.5 Stack Canaries — Protección contra Buffer Overflows

Los Stack Canaries son valores aleatorios que el compilador inserta en el stack entre las variables locales y la dirección de retorno, antes de que la función comience a ejecutarse. Al retornar, el sistema verifica que el canary no fue modificado. Si fue alterado, significa que hubo un overflow y el programa se aborta inmediatamente.

El nombre proviene de los canarios que los mineros llevaban a las minas para detectar gases tóxicos: el canario moría antes de que el gas fuera peligroso para los humanos, dando una señal de alerta temprana. El canary de stack funciona de la misma manera: detecta la corrupción antes de que el flujo de ejecución sea desviado.

¿Cómo funcionan los Stack Canaries?

- Antes de ejecutar una función, el sistema coloca un valor aleatorio (el canary) en la pila, entre las variables locales y la dirección de retorno.
- Al finalizar la función, antes de retornar, el programa compara el valor del canary con el original almacenado.
- Si el canary fue modificado, significa que hubo un buffer overflow: el programa se aborta con el mensaje: ***** stack smashing detected *****.
- Si el atacante intenta sobrescribir la dirección de retorno para ejecutar código malicioso, inevitablemente sobrescribirá el canary en el camino.



```
# Stack con canary protegido - layout de memoria

┌ Dirección de retorno ─ dirección alta
├ EBP guardado         ─ objetivo del atacante
├ CANARY = 0x7f3a91b2 (random) ─ valor aleatorio por ejecución
└ Variables locales / buf[N] ─ zona del overflow
                               ─ dirección baja

# Si el atacante hace overflow para llegar a la dirección de retorno,
# inevitablemente sobrescribe el canary en el camino.
# Al retornar, el sistema detecta la alteración:
*** stack smashing detected ***: ./programa terminated
Aborted (core dumped)

# Compilar con stack canary (activo por defecto en GCC moderno)
$ gcc -fstack-protector-all programa.c -o programa

# Verificar si el binario tiene canary
$ checksec --file=./programa | grep -i canary
Canary found
```

Sistemas como OpenSSH utilizan canaries para proteger su daemon sshd. Cuando el canary detecta una corrupción, el proceso es terminado inmediatamente, evitando que el atacante tome control del demonio.

Los canaries no protegen contra todos los overflows. Si el atacante puede leer el valor del canary (por ejemplo, a través de una vulnerabilidad de format string que lea el stack), puede copiarlo en su payload para que la verificación pase. Esto se llama canary leak. Por eso los canaries deben combinarse con ASLR y otras mitigaciones

1.6 KASLR — Kernel Address Space Layout Randomization

KASLR es la extensión de ASLR al espacio del kernel. Aleatoriza la dirección base donde se carga el kernel y sus módulos en cada arranque del sistema. Sin KASLR, un atacante que conoce la versión del kernel puede hardcodear en su exploit la dirección de estructuras críticas como la tabla de syscalls (sys_call_table) u otras estructuras internas del kernel.

```
# Verificar si KASLR está activo
$ cat /proc/cmdline | grep kaslr
... quiet splash kaslr ... ─ activo

# Sin KASLR: la dirección de la tabla de syscalls es constante
# El exploit puede hardcodear: sys_call_table = 0xffffffff81a001a0

# Con KASLR: la base del kernel cambia en cada arranque del sistema
```

```
$ sudo cat /proc/kallsyms | grep sys_call_table
ffffffff8c4001a0 R sys_call_table ← dirección diferente en cada boot

# Técnicas para evadir KASLR (más difícil que evadir ASLR de usuario):
# 1. Info leaks desde el kernel
#   Leer /proc/kallsyms si no está restringido por permisos
#   Aprovechar fugas de punteros de kernel en mensajes de error
# 2. Bruteforce (el espacio de randomización puede ser limitado)
#   En algunos sistemas de 32 bits el espacio es tan pequeño que
#   el bruteforce es factible en cuestión de segundos
```

1.7 ROP — Return-Oriented Programming

ROP (Return-Oriented Programming) es la técnica principal para evadir DEP/NX. En vez de inyectar código nuevo en el stack (bloqueado por NX), el atacante encadena fragmentos de código ya existente en el programa o sus bibliotecas. Cada fragmento, llamado gadget, termina en una instrucción ret. Al encadenarlos, el atacante construye un programa arbitrario usando únicamente código legítimo ya presente en el binario.

¿Cómo funciona ROP paso a paso?

- El atacante analiza el código de bibliotecas compartidas (como libc) para encontrar secuencias de instrucciones terminadas en ret. Estas son los gadgets.
- En lugar de ejecutar código malicioso inyectado, el atacante manipula el stack para que contenga las direcciones de los gadgets encadenados.
- Cuando la función vulnerable retorna, en vez de ir a la dirección de retorno legítima, va al primer gadget. Ese gadget ejecuta su código y el ret lo lleva al segundo gadget. Y así sucesivamente.
- El atacante ordena los gadgets de manera que realicen operaciones maliciosas, como llamar a system("/bin/sh") para obtener acceso a un shell.

```
# Gadgets típicos encontrados en libc:
0x7f3a1234: pop rdi ; ret ← carga el tope del stack en rdi
0x7f3a5678: pop rsi ; ret ← carga el tope del stack en rsi
0x7f3a9abc: call system ← llama a system()

# El atacante arma el stack así:

| addr de 'pop rdi; ret' | ← 1er gadget: carga el arg de system()
| dirección de '/bin/sh' | ← el string a pasar como arg
| addr de 'call system' | ← 2do gadget: llama a system()

# Resultado: se ejecuta system('/bin/sh') usando solo código de libc
# NX/DEP no puede bloquearlo porque no hay código inyectado
# El CPU ejecuta instrucciones legítimas de libc en un orden malicioso
```

Mitigaciones contra ROP: ASLR + KASLR (aleatoriza las direcciones de libc y del kernel, dificultando encontrar los gadgets), Control Flow Integrity (CFI, verifica que las llamadas a funciones sigan un flujo predecible), y la combinación de todas las defensas en capas.

1.8 CFI — Control Flow Integrity

Control Flow Integrity (CFI) es una mitigación de nivel de compilador que verifica en tiempo de ejecución que las llamadas a funciones y los retornos siguen un flujo de control legítimo definido en tiempo de compilación. Su objetivo es detectar y bloquear ataques de tipo ROP que manipulan el flujo de ejecución.

- CFI analiza el programa en tiempo de compilación y construye un grafo de flujo de control válido.
- En tiempo de ejecución, antes de cada llamada indirecta (call o ret), verifica que el destino sea un objetivo legítimo según el grafo.
- Si el atacante modifica el stack para hacer que un ret vaya a un gadget de ROP, CFI detecta que ese destino no es un retorno válido y aborta el programa.

```
# Compilar con CFI en Clang
$ clang -fsanitize=cfi -flto -fvisibility=hidden programa.c -o programa_cfi

# GCC: implementación parcial de CFI con -fstack-protector-strong
$ gcc -fstack-protector-strong -fcf-protection=full programa.c -o programa

# Intel CET (Control-flow Enforcement Technology) — hardware CFI
# Introduce instrucciones ENDBRANCH y shadow stack para verificar retornos
```

Seguridad del Kernel y Protección de Syscalls

La seguridad del kernel es fundamental en los sistemas operativos, ya que cualquier vulnerabilidad en este nivel puede otorgar a un atacante control total del sistema. Las estrategias clave incluyen la protección del espacio del kernel, el hardening de syscalls y el monitoreo seguro mediante tecnologías como Secure Boot, Seccomp-BPF y eBPF.

2.1 Protección del Espacio del Kernel

El kernel opera en un espacio de memoria privilegiado que debe mantenerse aislado del espacio de usuario para prevenir escalaciones de privilegios y modificaciones maliciosas. Dos mecanismos esenciales para esta protección son Secure Boot y la restricción de acceso a dispositivos de memoria críticos.

Secure Boot

Secure Boot, parte del estándar UEFI, garantiza que solo código firmado digitalmente sea ejecutado durante el arranque del sistema. Esto evita la carga de módulos del kernel o controladores no autorizados, mitigando ataques como los rootkits de arranque (bootkits).

- Impide la ejecución de código malicioso en el arranque del sistema.
- Solo permite cargar componentes firmados digitalmente: firmware UEFI, bootloader, kernel, módulos del kernel.
- Si un atacante intenta cargar un módulo del kernel sin firma, Secure Boot lo rechaza.



```
# Verificar el estado de Secure Boot
$ mokutil --sb-state
SecureBoot enabled ← está activo

# Ver los certificados de Secure Boot cargados
$ mokutil --list-enrolled

# Intentar cargar un módulo sin firma con Secure Boot activo:
insmod modulo_no_firmado.ko
insmod: ERROR: could not insert module: Required key not available
```

Secure Boot protege el proceso de arranque, pero no protege contra ataques que ocurren después del arranque. Un atacante con acceso root puede intentar modificar el kernel en tiempo de ejecución a través de otros vectores. Para eso existen Kernel Lockdown y los módulos firmados.

Restricción de Acceso a /dev/mem y /dev/kmem

Los dispositivos /dev/mem y /dev/kmem permiten acceso directo a la memoria física del sistema y a la memoria del kernel respectivamente. Históricamente fueron explotados para modificar estructuras críticas del kernel en tiempo de ejecución, sin necesidad de cargar un módulo malicioso.

- /dev/mem: acceso directo a la memoria RAM física. Un proceso con acceso puede leer y modificar cualquier región de memoria del sistema.
- /dev/kmem: acceso directo al espacio de memoria virtual del kernel. Permite modificar estructuras internas del kernel en tiempo de ejecución.

```
# Restringir acceso a /dev/mem (solo root puede acceder)
$ sudo chmod 600 /dev/mem

# Kernel compilado con CONFIG_STRICT_DEVMEM=y
# bloquea el acceso a regiones críticas del espacio de kernel

# Verificar la configuración del kernel al respecto
$ grep CONFIG_STRICT_DEVMEM /boot/config-$(uname -r)
CONFIG_STRICT_DEVMEM=y ← protección activa

# Un ataque clásico sin estas restricciones:
# usar /dev/mem para modificar la tabla de syscalls directamente
# en tiempo de ejecución → redirigir sys_read() a código malicioso
# Con CONFIG_STRICT_DEVMEM=y este ataque es bloqueado
```

2.2 Hardening de Llamadas al Sistema (Syscalls)

Las llamadas al sistema (syscalls) son la principal interfaz entre los procesos de usuario y el kernel. Constituyen el puente que permite a los programas acceder a recursos de hardware, gestionar procesos, leer archivos y comunicarse por red. Restringir su uso es fundamental para reducir la superficie de ataque.

Syscalls Nativas vs. Syscalls en Virtualización

Syscalls Nativas: son llamadas al sistema que interactúan directamente con el núcleo del sistema operativo. Funcionan como puente entre el espacio de usuario y el espacio del kernel. Cuando un proceso llama a `open()`, el sistema la traduce en la syscall `sys_open()`, que accede al sistema de archivos del kernel directamente.

Syscalls en Virtualización o Emulación: en entornos virtualizados o emulados, las syscalls no interactúan directamente con el kernel del sistema host. Una capa de abstracción maneja las llamadas, simulando el comportamiento del sistema operativo real. Por ejemplo, si un programa ejecuta `open()` dentro de QEMU, la llamada no llega al kernel del host — el emulador la intercepta y la procesa dentro del sistema virtualizado.

Los ataques de inyección de código utilizan syscalls como `ptrace()`, `mprotect()` o `execve()` para modificar procesos de alto privilegio. Herramientas como `seccomp-BPF` permiten filtrar exactamente qué syscalls puede ejecutar cada proceso, limitando drásticamente los vectores de ataque disponibles.

2.3 Seccomp-BPF — Filtrado de Syscalls

Seccomp (Secure Computing Mode) es una tecnología del kernel Linux que permite restringir las syscalls que un proceso puede ejecutar, aplicando filtros definidos en BPF (Berkeley Packet Filter). Su uso es clave en entornos como contenedores Docker y navegadores web (Chrome) para evitar ejecuciones no autorizadas.

Cuando un proceso intenta ejecutar una syscall que no está en la lista permitida por su perfil seccomp, el kernel puede: terminar el proceso (SIGKILL), retornar un error (ERRNO), o en modo strict, matar el proceso inmediatamente.

```
# Docker aplica perfiles seccomp por defecto
# Ejecutar con perfil seccomp personalizado:
$ docker run --security-opt seccomp=/path/to/perfil.json ubuntu

# Las syscalls bloqueadas por defecto en Docker incluyen:
# keyctl, add_key, request_key (acceso al keyring del kernel)
# ptrace (inyección en otros procesos)
# syslog (acceso a logs del kernel)
# kexec_load (cargar un nuevo kernel)

# Verificar el perfil seccomp de un proceso en ejecución
$ cat /proc/$(pidof proceso)/status | grep Seccomp
Seccomp: 2    ← 0=desactivado, 1=strict, 2=filter (BPF activo)

# En código C - activar seccomp desde la aplicación:
# #include <seccomp.h>
# scmp_filter_ctx ctx = seccomp_init(SCMP_ACT_KILL);
# seccomp_rule_add(ctx, SCMP_ACT_ALLOW, SCMP_SYS(read), 0);
# seccomp_rule_add(ctx, SCMP_ACT_ALLOW, SCMP_SYS(write), 0);
# seccomp_load(ctx); // solo read() y write() permitidos
```

Google Chrome usa seccomp para aislar los procesos de renderizado. Cada pestaña corre en un proceso separado con un perfil seccomp muy restrictivo. Si un atacante explota una vulnerabilidad en el motor de JavaScript (V8), solo puede ejecutar las syscalls permitidas por el perfil — no puede, por ejemplo, llamar a `execve()` para iniciar otros procesos o usar `ptrace()` para inyectar código en procesos del sistema.

2.4 eBPF — Extended Berkeley Packet Filter

eBPF (Extended Berkeley Packet Filter) es una tecnología avanzada que permite ejecutar programas dentro del kernel de forma segura y eficiente, sin necesidad de modificar su código fuente ni reiniciar el sistema. Originalmente diseñado para el filtrado de paquetes de red, eBPF ha evolucionado hasta convertirse en una herramienta poderosa para monitorear y proteger el kernel en tiempo real.

A diferencia de los módulos del kernel tradicionales (que requieren reinicio y pueden desestabilizar el sistema), los programas eBPF son verificados por el kernel antes de ejecutarse, garantizando que no puedan bloquear el sistema ni causar corrupciones.

Casos de uso de eBPF en seguridad

- Monitorear syscalls sospechosas para detectar intentos de explotación o escalación de privilegios.
- Bloquear acceso a archivos críticos del sistema como `/etc/shadow` o `/etc/passwd`.
- Detectar modificaciones en memoria que puedan indicar la presencia de rootkits o malware en el kernel.
- Auditar eventos del kernel en tiempo real sin afectar el rendimiento del sistema.
- Generar alertas cuando un proceso inesperado intenta usar syscalls peligrosas como `execve()` o `ptrace()`.

```
# Ejemplo: monitorear intentos de ejecución de binarios sospechosos
# con bpftrace (herramienta de scripting sobre eBPF)

# Monitorear todas las llamadas a execve() en tiempo real:
$ sudo bpftrace -e 'tracepoint:syscalls:sys_enter_execve { printf("%s -> %s\n",
comm, str(args->filename)); }'
bash -> /usr/bin/ls
bash -> /usr/bin/id
bash -> /bin/sh           ← intento sospechoso de abrir una shell

# Monitorear accesos a /etc/shadow (archivo de contraseñas):
$ sudo bpftrace -e 'tracepoint:syscalls:sys_enter_openat { if (str(args->filename)
== "/etc/shadow") printf("Acceso a shadow por: %s (PID %d)\n", comm, pid); }'
Acceso a shadow por: python3 (PID 12345) ← alerta
```

Herramientas como Cilium (seguridad de red en Kubernetes) y Falco (detección de amenazas en contenedores) están construidas sobre eBPF. La ventaja clave: se puede agregar monitoreo de seguridad avanzado a un sistema en producción sin reiniciarlo ni modificar el kernel.

Ataques Avanzados y Mitigaciones del Kernel

3.1 Rootkits a Nivel de Kernel

Los rootkits a nivel de kernel son una de las amenazas más peligrosas porque operan en el mismo nivel de privilegio que el sistema operativo. Pueden modificar estructuras críticas del sistema operativo para ocultar procesos, escalar privilegios, interceptar syscalls y mantener acceso persistente en un sistema comprometido — todo mientras evaden las herramientas de detección estándar.

¿Cómo funcionan los rootkits de kernel?

- Carga de módulo malicioso: el rootkit se carga como un módulo del kernel legítimo (usando insmod). Una vez dentro del espacio del kernel, tiene acceso total al sistema.
- Hooking de syscalls: modifica la tabla de syscalls (sys_call_table) para redirigir llamadas como getdents() (listar archivos) o kill() a versiones maliciosas que ocultan los procesos del atacante.
- Ocultamiento de procesos: al interceptar getdents64(), el rootkit puede filtrar los procesos maliciosos de la lista antes de devolvérsela al usuario. Comandos como ps o top no los muestran.
- Ocultamiento de módulos: el rootkit se elimina de la lista de módulos del kernel (visible con lsmod) para que su presencia no sea detectada.

Diamorphine es un rootkit real de código abierto disponible en GitHub (para propósitos educativos) que implementa todas estas técnicas. Manipula la tabla de syscalls para hacer que procesos maliciosos sean invisibles y evitar su detección. Fue usado como base para entender cómo funcionan los rootkits de kernel modernos.

Mitigaciones contra rootkits de kernel

Uso de módulos firmados

- Configurar el kernel con CONFIG_MODULE_SIG=y para permitir solo la carga de módulos con firmas digitales verificadas.
- Con esta configuración, insmod rechaza cualquier módulo que no esté firmado por una clave de confianza del sistema.
- Esto evita que atacantes inserten módulos maliciosos que alteren el comportamiento del kernel.

```
# Verificar si los módulos firmados están habilitados
$ grep CONFIG_MODULE_SIG /boot/config-$(uname -r)
CONFIG_MODULE_SIG=y
CONFIG_MODULE_SIG_FORCE=y ← rechaza módulos sin firma

# Intentar cargar un módulo sin firma con MODULE_SIG_FORCE:
insmod rootkit.ko
insmod: ERROR: could not insert module: Required key not available
```

Kernel Lockdown Mode

Habilitar LOCKDOWN_LSM restringe el acceso a interfaces sensibles del kernel, incluso para procesos con privilegios de root. Existen dos modos:

- Modo integrity: restringe operaciones que podrían modificar el kernel en ejecución, como /dev/mem, kexec_load y módulos sin firma.
- Modo confidentiality: además del modo integrity, bloquea operaciones que podrían extraer información del kernel, como la lectura de secretos del kernel o el acceso a memoria del kernel por parte de software de monitoreo no firmado.

```
# Activar Kernel Lockdown (requiere reinicio con parámetro)
# Agregar al bootloader: lockdown=integrity o lockdown=confidentiality

# Verificar el estado actual de Lockdown
$ cat /sys/kernel/security/lockdown
[none] integrity confidentiality ← 'none' está activo actualmente

# Cambiar a modo integrity en tiempo de ejecución (si está permitido):
$ echo integrity | sudo tee /sys/kernel/security/lockdown
```

Monitoreo con eBPF

Implementar programas eBPF que supervisen la actividad del kernel y detecten modificaciones sospechosas en estructuras internas. Por ejemplo, detectar si algún proceso intenta modificar la sys_call_table o si se carga un módulo inesperado.

3.2 Caso Real — CVE-2022-2588

CVE-2022-2588 es un ejemplo real de vulnerabilidad crítica en el kernel Linux, descubierta en 2022. Afectaba al subsistema netfilter (el firewall del kernel) y permitía a un atacante local ejecutar código arbitrario con privilegios de root a través de la explotación de un use-after-free en la gestión de reglas de filtrado de red.

- Tipo de vulnerabilidad: use-after-free en el subsistema cls_route del kernel.
- Impacto: escalación de privilegios local — un usuario sin privilegios podía obtener root.
- Versiones afectadas: Linux kernel desde 4.14 hasta 5.18.
- CVSS Score: 7.8 (High) — explotable localmente, sin interacción del usuario.

```
# CVE-2022-2588 - cls_route use-after-free

# El fallo estaba en cómo el kernel liberaba y reutilizaba
# estructuras de memoria en el módulo de filtrado de rutas (cls_route4)

# Un atacante local podía:
# 1. Crear una regla de netfilter que apunte a una estructura en memoria
# 2. Liberar esa estructura (free)
# 3. Reclamar esa memoria con datos controlados por el atacante
# 4. La regla de netfilter ahora apunta a datos del atacante
```



```
# 5. Ejecutar código arbitrario en el contexto del kernel

# Mitigación: actualizar el kernel a versión parcheada
$ uname -r
5.19.0-41-generic ← versión con el parche aplicado

# Verificar si el sistema tiene el parche:
$ grep -r 'CVE-2022-2588' /var/log/dpkg.log | tail -3
```

CVE-2022-2588 ilustra por qué la seguridad del kernel importa incluso en sistemas bien administrados. El atacante no necesitaba acceso remoto ni privilegios especiales — solo necesitaba una cuenta de usuario local. Las defensas en capas (KASLR, módulos firmados, Lockdown) dificultan la explotación pero no reemplazan la actualización del kernel

Textos de referencia del curso

- OSTEP, Cap. 13 (Memory API): Asignación y protección de memoria.
- OSTEP, Cap. 39 (Security): Mitigaciones como ASLR, NX Bit, Stack Canaries y protección del kernel.
- APUE, Cap. 8 (Processes): Manejo de memoria de procesos y seguridad.
- APUE, Cap. 2 (System Calls and Library Functions): Seguridad en syscalls de UNIX/Linux.
- Silberschatz, Cap. 14 (Protection): Control de acceso en el kernel.